The AI Behind LinkedIn Recruiter search and recommendation systems

Qi Guo April 22, 2019

Share Tweet Share

Co-authors: Qi Guo, Sahin Cem Geyik, Cagri Ozcaglar, Ketan Thakkar, Nadeem Anjum, and Krishnaram Kenthapadi

LinkedIn Talent Solutions serves as a marketplace for employers to reach out to potential candidates and for job seekers to find career opportunities. A key mechanism to help achieve these goals is the LinkedIn Recruiter product, which helps recruiters and hiring managers source suitable talent and enables them to identify "talent pools" that are optimized for the likelihood of making a successful hire. In this blog post, we will first highlight a few unique information retrieval, system, and modeling challenges associated with talent search and recommendation systems. We will then describe how we formulated and addressed these challenges, the overall system design and architecture, the issues encountered in practice, and the lessons learned from the production deployment of these systems at LinkedIn.

Introduction

The LinkedIn Recruiter product provides a ranked list of candidates corresponding to a search request in the form of a query, a job posting, or a recommended candidate. Given a search request, candidates that match the request are selected and then ranked based on a variety of factors (such as the similarity of their work experience/skills with the search criteria, job posting location, and the likelihood of a response from an interested candidate) using machine-learned models in multiple passes. A screenshot from the application is presented in Figure 1.

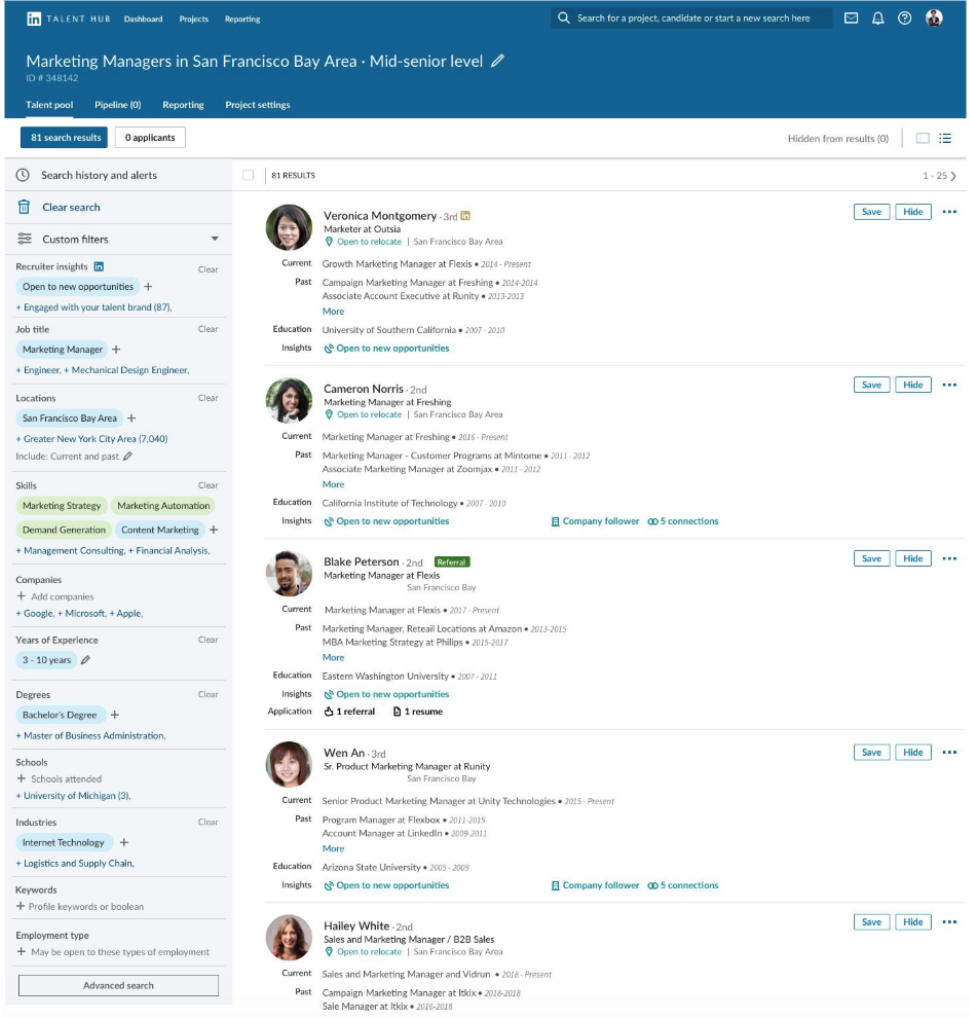

Figure 1: A (mocked) screenshot from the LinkedIn Recruiter product

For each recommended candidate, the recruiter can perform the following actions:

- View the candidate's profile,
- Save the profile to a hiring project (as a potential fit), and,
- Send an InMail to the candidate.

In this blog post, we discuss multiple methodologies we utilize in the talent search systems at LinkedIn. These methods aim to address a set of unique information retrieval challenges associated with talent search and recommendation systems, which can be listed as the following:

1. Unlike traditional search and recommendation systems, which solely focus on estimating how relevant an item is for a given query, the talent search domain requires mutual interest between the recruiter and the candidate in the context of the job opportunity. In other words, we require not just that a candidate shown must be relevant to the recruiter's query, but also that the candidate contacted by the recruiter must show interest in the job opportunity. Hence, it is crucial to use appropriate metrics for model optimization, as well as for online A/B testing. We define a new objective, InMail Accept, which occurs when a candidate received an InMail from a recruiter and replies with a positive response. We take the InMail accept as an indication of two-way interest, which may lead to the candidate

candidates that received and accepted an InMail (viewed as precision@k) as the main evaluation measure when we experiment with new models for the Recruiter product (please see our BigData'15, CIKM'18, and SIGIR'18 papers for details).

2. Additionally, the underlying query to the talent search system could be quite complex, combining several structured fields, such as canonical title(s), canonical skill(s), and company name, along with unstructured fields, such as free-text keywords. Depending on the application, the query could either consist of an explicitly entered query text and selected facets (Recruiter Search or "Talent Search" in the research literature), or be implicit in the form of a job opening or ideal candidate(s) for a job (Recommended Matches). In Recruiter Search, to assist our users with query formulation, we also suggest related entities that the user might be interested in, e.g., recommending titles like "Data Scientist" and skills like "Data Mining" to recruiters searching for title "Machine Learning Engineer." With a given query, our goal is to determine a ranked list of the most relevant candidates in real time among hundreds of millions of semi-structured candidate profiles. Consequently, robust standardization, intelligent query understanding and query suggestion, scalable indexing, high-recall candidate selection, effective ranking algorithms, and efficient multi-pass scoring/ranking systems are essential (please see our SIGIR'16 and WWW'16 papers for details).

3. Finally, personalization is of the essence to a talent search system where we need to model the intents and preferences of recruiters concerning the type of candidates they are looking for. This could either be achieved through offline learning of personalized models through stored recruiter usage data, or via understanding the preferences of the recruiter during online use. On occasion, the recruiter may not even be sure about, say, the set of skills to search for, and this has to be learned through a set of candidate recommendation and evaluation stages (please see our CIKM'18 and WWW'19 papers for more details on how we apply personalization for talent search at LinkedIn).

With the help of the modeling approaches described in this blog post, we have been able to steadily increase our key business metrics. For example, in a two-year period we were able to double the number of InMails accepted by job candidates. All these efforts also contribute to our vision of creating economic opportunity for every member of the global workforce.

Methodologies

Non-linear modeling with Gradient Boosted Decision Trees

Our earliest machine learning model for LinkedIn Recruiter search ranking was a linear model. Linear models are the easiest to debug, interpret, and deploy, and thus a good choice in the beginning. But linear models cannot capture non-linear feature interactions well. We now use Gradient Boosted Decision Trees (GBDT) to unleash the power of our data. GBDT models feature interaction explicitly through a tree structure. Aside from a larger hypothesis space, GBDT has a few other advantages, like working well with feature collinearity, handling features with different ranges and missing feature values, etc. Our online experiments with GBDTs for Recruiter Search

ranking were able to achieve statistically significant high single-digit percentage improvement over engagement (between the recruiter and the candidate) metrics.

Context-aware ranking with pairwise learning-to-rank

To add awareness of the search context to our GBDT models, we worked on the following improvements. For searcher context, we added some personalization features. For query context, we added more query-candidate matching features, some directly leveraged from LinkedIn's flagship search product. And very importantly, we used GBDT models with a pairwise ranking objective, to compare candidates within the same context, i.e., the same search request. Pairwise optimization compares pairs of impressions within the same search query. Pointwise optimization assumes all the impressions are independent, no matter if they are in the same search query or not. For this reason, pairwise ranking is more aware of the context. Application of contextual features and pairwise GBDT models helped us achieve a low two-digit (in the tens) percentage improvement in the recruiter-candidate engagement metrics.

Deep and representation learning efforts

As discussed above, the existing ranking system powering the LinkedIn Recruiter product utilizes a Gradient Boosted Decision Tree (GBDT) model due to its advantages over linear models. While GBDT provides quite a strong performance, it poses the following challenges:

1. It is quite non-trivial to augment a tree ensemble model with other trainable components, such as embeddings for discrete features. Such practices typically require joint training of the model with the component/feature, while the tree ensemble model assumes that the features themselves need not be trained.

2. Tree models do not work well with sparse id features such as skill ids, company ids, and member ids that we may want to utilize for talent search ranking. Since a sparse feature is non-zero for a relatively small number of examples, it has a small likelihood of being chosen by the tree generation at each boosting step, especially since the learned trees are shallow in general.

3. Tree models lack flexibility in model engineering. It might be desirable to use novel loss functions, or augment the current objective function with other terms. Such modifications are not easily achievable with GBDT models, but are relatively straightforward for deep learning models based on differentiable programming. A neural network model with a final (generalized) linear layer also makes it easier to adopt approaches such as transfer learning and online learning.

In order to overcome these challenges, we also explored the usage of neural network based models, which provide sufficient flexibility in the design and model specification. Our offline experiments with pairwise deep-models of up to three layers have shown promise against our baseline GBDT model, where we have observed low single-digit improvements over baseline engagement metrics.

We're currently exploring utilizing LinkedIn's recent model serving infrastructure improvements to deploy neural network models.

Another significant challenge for Talent Search modeling pertains to the sheer number of available entities that a recruiter can include as part of their search, and how to utilize them for candidate selection as well as ranking. For example, the recruiter can choose from tens of thousands of LinkedIn's standardized skills. Since different entities could be related to each other (to a varying degree), using syntactic features (e.g., fraction of query skills possessed by a candidate) has its limitations. Instead, it is more desirable to utilize semantic representations of entities—for example, in the form of low dimensional embeddings. Such representations allow for numerous sparse entities to be better incorporated as part of a machine learning model. Within Recruiter, we utilize unsupervised network embeddings trained via the Large-Scale Information Network Embeddings (LINE) approach. LINE can optimize for first-order proximity and second-order proximity, is applicable to both directed and undirected graphs, and can scale well. The network embeddings are trained with a modified version of the LinkedIn Economic Graph, where we generate edge weights between entities according to the number of LinkedIn members that have the two entities listed together in their profile (e.g., they have both of the skills or worked at both companies on two ends of the edges, etc.). An illustration for the graph in terms of company entities is given below:

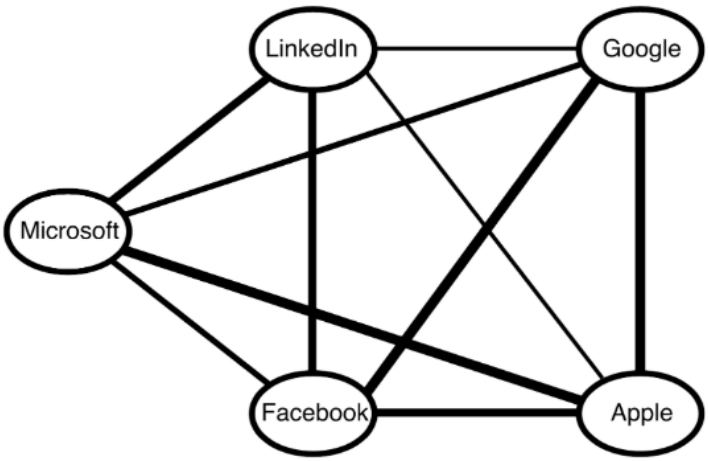

Figure 2: An illustration of the company entity graph

We have utilized the generated embeddings as part of the features on which we train GBDT models. Our online experiments of a GBDT model with network embedding semantic similarity features have shown low single-digit improvements over engagement metrics. The ranking lift, however, was not statistically significant. The hypothesis is that, because the retrieval process is doing exact match based on title ids, the embedding-based similarity won't differentiate the retrieved results by much. This motivated us to apply this to the retrieval stage. We implemented a query expansion strategy that adds results with semantically similar titles, like "Software Developer" for "Software Engineer," when the number of returned results from the original query is too small.

Entity-level personalization with GLMix

In the Recruiter Search domain, multiple entities, such as recruiters, contracts,

model. For entity-level personalization, we used Generalized Linear Mixed (GLMix) models, and experimented with personalization for multiple entities in the Recruiter Search domain. In order allow nonlinear feature interactions, we used a GBDT model in production as a feature transformer, which generates tree interaction features and a GBDT model score. Based on our offline experiments, we used recruiter-level and contract-level personalization in the final GLMix model. Figure 3 shows the pipeline for building GLMix models using learning-to-rank features, tree interaction features, and GBDT model scores. Learning-to-rank features are used as input to a pre-trained GBDT model, which generates tree ensembles that are encoded into tree interaction features and a GBDT model score for each data point. Then, using the original learning-to-rank features and their nonlinear transformations in the form of tree interaction features and GBDT model scores, we build a GLMix model with recruiter-level and contract-level personalization.

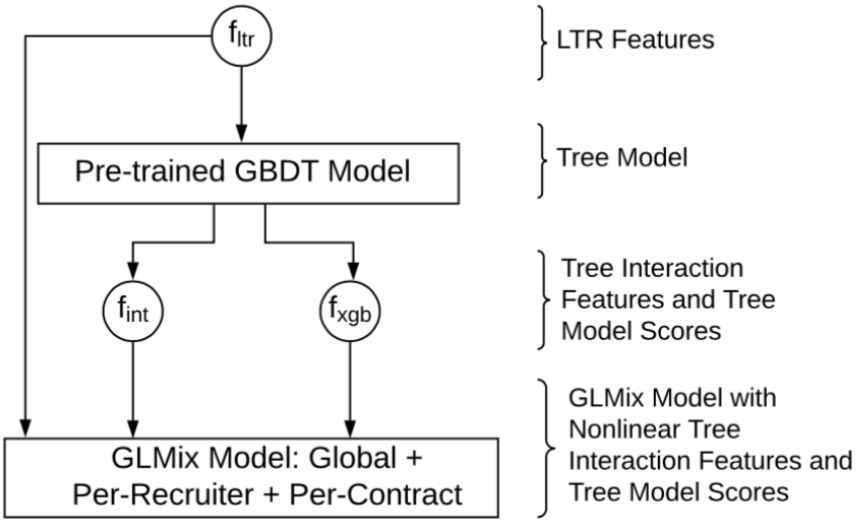

Figure 3: Pipeline for GLMix models with tree interaction features

In online experiments, we benchmarked the best GLMix model variant, GLMix global + per-contract + per-recruiter model, with the production model at the time, which was a pairwise GBDT model. Online experiment results utilizing the GLMix model with tree interaction features resulted in low single-digit statistically significant improvements of engagement metrics, compared to the baseline pairwise GBDT model.

In-session online personalization

A shortcoming of utilizing offline-learned models is the fact that, as the recruiter examines the recommended candidates and provides feedback, that feedback is not taken into account during the current search session. Within the Recruiter team, we have therefore worked on systems that adapt to the user's feedback and, after some steps (i.e., immediate feedback given to candidates that are presented one at a time), recommend the best candidates for the job.

Below is the architecture we utilize for such a system, which first separates the potential candidate space for the job into skill groups. Then, a multi-armed bandit model is utilized to understand which group is more desirable based on the recruiter's

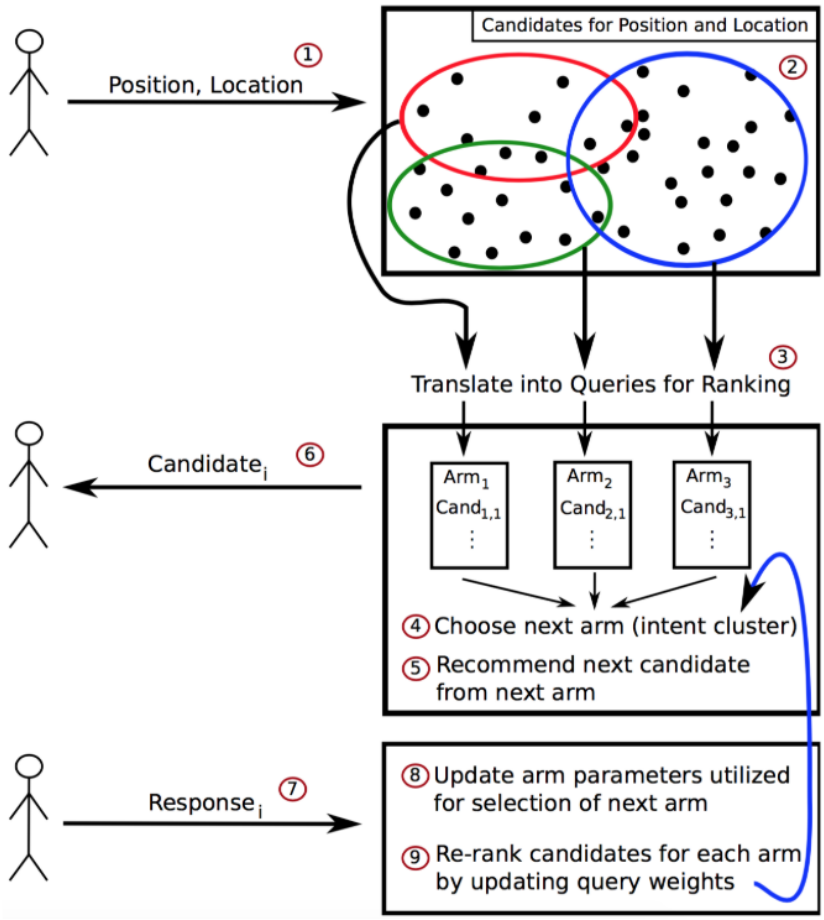

Figure 4: The architecture of the multi-armed bandit online personalization system

Below are some results from our initial experimentation with such a recommendation algorithm. We present the precision (in terms are whether candidates are positively rated by the user) of the recommendations as more candidates are presented to the user. The graph shows that the quality of the recommended candidates improve (we get more and more positive feedback), as more feedback is incorporated to the recommendation model. The exact percentage of candidates that were marked as a good match was modified for company policy.

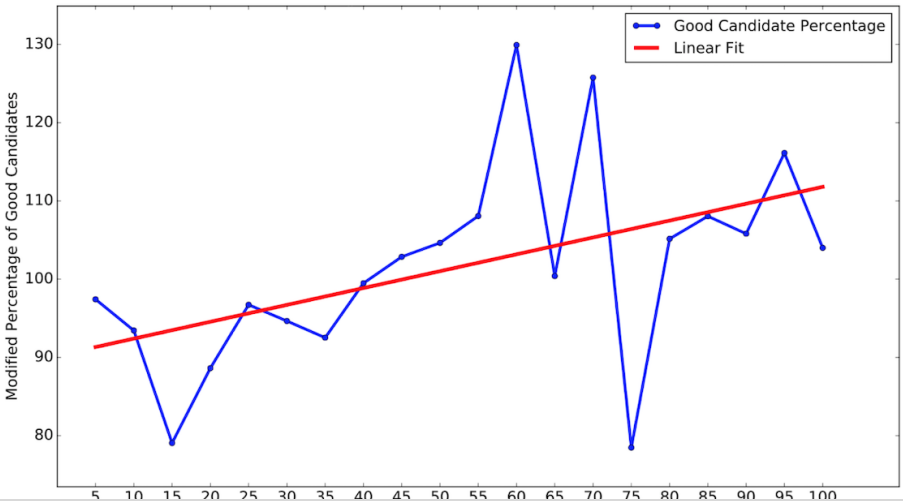

Figure 5: Percentage of good-match candidates at each index

Another recent effort in the online learning direction within our team has been toward learning a variety of profile attributes (e.g., skill, title, industry, and seniority) that might be most relevant to the recruiter based on the feedback for each candidate with those attributes. If a recruiter is consistently interested in candidates who are, say, accountants with leadership skills, or project managers who are adept at social media, we aim to recommend more of such candidates, implicitly learning a search query for the current intent of the recruiter. This all happens online in real time so that the feedback is taken instantly into account. For more details on this methodology, we invite the interested readers to check out another recent LinkedIn Engineering blog post.

System Design and Architecture

LinkedIn has built a search stack on top of Lucene called Galene, and contributed to various plug-ins, including capability to live-update search index. The search index consists of two types of fields:

- **The inverted field:** a mapping from search terms to the list of entities (members) that contain them.
- **The forward field:** a mapping from entities (members) to metadata about them.

These search index fields contribute to the evaluation of machine learning feature values in search ranking. The freshness of data in the search index fields is also of high importance for machine learning features.

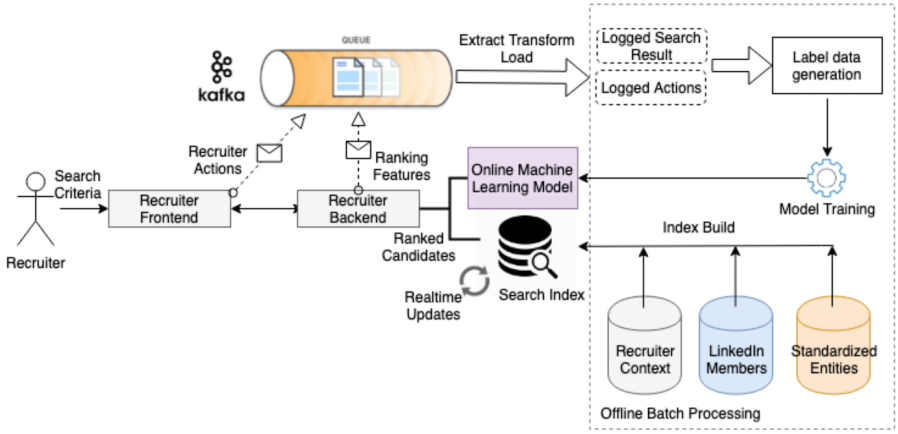

Figure 6: Talent Search architecture and flow

Recruiter Search has a layered ranking architecture:

- **L1:** Scoops into the talent pool and scores/ranks candidates. In this layer, candidate retrieval and ranking are done in a distributed fashion.
- **L2:** Refines the short-listed talent to apply more dynamic features using external caches.

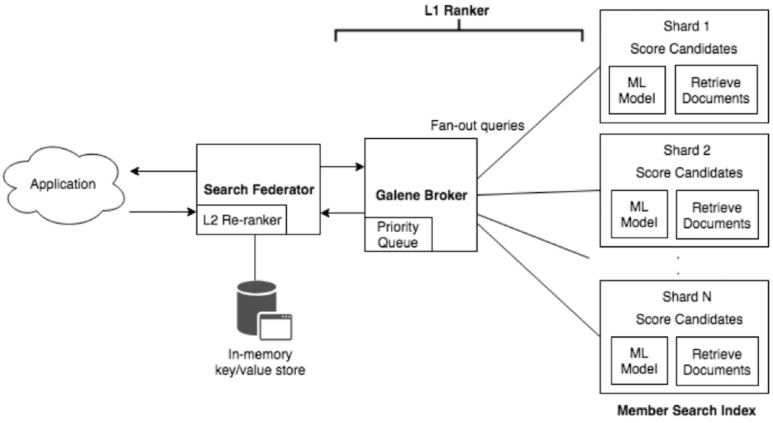

Figure 7: Detailed search retrieval and ranking architecture

The Galene broker system fans out the search query request to multiple search index partitions. Each partition retrieves the matched documents and applies the machine learning model to retrieved candidates. Each partition ranks a subset of candidates, then the broker gathers the ranked candidates and returns them to the federator. The federator further ranks the retrieved candidates using additional ranking features that are dynamic or referred to from the cache—this is the L2 ranking layer. For more details about our federated search architecture, please see the prior LinkedIn Engineering blog post related to the topic.

Conclusion

In this blog post, we have given a brief overview of the journey of our model explorations and the architecture utilized for Talent Search systems at LinkedIn. As shown, these models have had an impact on our key business metrics. More importantly, however, is the improvement in outcomes for our members and customers. Recently, motivated by LinkedIn's goal of creating economic opportunity for every member of the global workforce and by a keen interest from our customers in making sure that they are able to source diverse talent, we have also deployed gender representative ranking as part of our Talent Search systems to all users of the LinkedIn Recruiter product worldwide. We look forward to sharing additional insights into the evolution of AI for recruiting at LinkedIn.

Acknowledgments

This blog post is based on the ACM SIGIR 2018 paper (slides) on talent search and recommendation systems at LinkedIn, an ACM CIKM 2018 paper on in-session personalization for talent search, an ACM CIKM 2018 paper on deep and representation learning for talent search, and a WWW 2019 paper on entity-personalized talent search models with tree interaction features. We would like to thank all members of the LinkedIn Talent Solutions team for their collaboration in deploying our work as part of the launched products, and the following people for insightful feedback and discussions (alphabetically ordered): Abhishek Gupta, Ajit Singh, Alex Shelkovnykov, Alexandre Patry, Alice Wu, Anish Nair, Anthony Hsu, Aparna Krishnan, Arashpreet Singh Mor, Ashish Gupta, Badrul Sarwar, Baolei Li, Bo Hu, Brian Schmitz, Busheng Wu, Chang-Ming Tsai, Dan Liu, Daniel Hewlett, David

Gil Cottle, Gio Borje, Gungor Polatkan, Gurwinder Gulati, Haitong Tian, Hakan Inan, Hong Li, Huiji Gao, Janardhanan Vembunarayanan, Jared Goralnick, Jieqing Dai, Joel Young, Jonathan Pohl, Joshua Hartman, Jun Xie, Keqiu Hu, Kexin Fei, Lei Ni, Lin Yang, Luthfur Chowdhury, Luxin Kang, Mathuri Vasudev, Matthew Deng, Meng Meng, Michael Chernyak, Mingzhou Zhou, Patrick Cheung, Prakhar Sharma, Qinxia Wang, Rakesh Malladi, Ram Swaminathan, Rohan Ramanath, Runfang Zhou, Ryan Smith, Sai Krishna Bollam, Sara Smoot, Scott Banachowski, Sen Zhou, Shan Zhou, Shipeng Yu, Siyao Sun, Skylar Payne, Sriram Vasudevan, Tanvi Motwani, Tian Lan, Viet Ha-Thuc, Vijay Dialani, Wei Lu, Wen Pu, Wensheng Sun, Wenxiang Chen, Xianren Wu, Xiaoyi Zhang, Xuebin Yan, Xuhong Zhang, Yan Yan, Yen-Jung Chang, Yi Guo, Yiming Ma, Yu Gong, and Zian Yu.

Topics

artificial intelligence, machine learning, research, Product Design, Data

Related story
Production testing with dark canaries

Related story
Bridging batch and stream processing for the Recruiter usage statistics dashboard

LinkedIn Corporation © 2021 About Cookie Policy Privacy Policy User Agreement Accessibility